

02 · Instalación y ejecución

Documentación de sistema

Versión analizada: **0.1.1+2**

Commit: **a7ebf2c**

Fecha del documento: **2026-08-27**

Documento generado desde el Markdown de [docs/system-documentation/](#) con [tool/build_system_documentation_pdf.py](#). El Markdown es la fuente: no editar este PDF.

Contenido

1. Requisitos previos
2. Por qué hay un paso de generación
3. Instalación desde cero
4. Variables de entorno
5. Configuración inicial
6. Creación o restauración de la base de datos
7. Ejecución en desarrollo
8. Ejecución en producción
9. Ejecución de las pruebas
10. Verificaciones offline
11. Gate completo, igual que la CI
12. Errores frecuentes durante la instalación
13. Herramientas auxiliares

Todos los comandos de este documento se ejecutan desde la raíz del repositorio. Ninguno contiene secretos reales.

Requisitos previos

Requisito	Versión	Por qué esa versión	Obligatorio
Flutter	3.44.7	Fijada en <code>../.fvmrc</code> y en los tres workflows de CI. <code>pubspec.yaml</code> exige <code>>=3.44.0</code>	Sí
Dart	<code>>=3.12.0 <4.0.0</code>	Declarado en <code>pubspec.yaml</code> . Viene con Flutter	Sí
Python	3.12	Lo usan las 13 herramientas de <code>tool/</code>	Sí
uv	0.12.5	Solo lo usa la CI para inyectar <code>pyyaml</code> sin instalarlo. En local basta con tener <code>pyyaml</code>	No
Android SDK	API 24 o superior	<code>tool/bootstrap.py</code> fija <code>minSdk = 24</code>	Para Android
Xcode + macOS	—	Solo un host macOS puede generar y compilar el target iOS	Para iOS
Pillow	cualquiera	Solo para regenerar los iconos. Compilar no lo necesita	No

Comprobación rápida del entorno:

```
flutter --version
python --version
```

Por qué hay un paso de generación

Las carpetas `android/`, `ios/` y `web/` no están en el repositorio. `.gitignore` las excluye a propósito. Se generan de forma reproducible con `tool/bootstrap.py`, que además aplica una lista de parches necesarios: permisos, iconos, `minSdk`, versiones de la cadena Gradle, textos de uso de cámara y Face ID, y el cambio de `FlutterActivity` a `FlutterFragmentActivity` que exige la biometría.

Consecuencia práctica: **un clon recién hecho no compila hasta ejecutar el bootstrap.**

Instalación desde cero

```
git clone https://github.com/vladimiracunadev-create/rootcause-qr-inspector.git
cd rootcause-qr-inspector
flutter pub get
python tool/bootstrap.py --platforms android,web
```

`--platforms` acepta `android`, `ios` y `web`, separados por comas. Sin el argumento genera `android` (y también `ios` si el sistema es macOS). El target `web` existe solo como canal de demostración; no es una plataforma del producto.

En macOS, para trabajar con iOS:

```
python tool/bootstrap.py --platforms ios
```

Para regenerar desde cero una carpeta ya existente:

```
python tool/bootstrap.py --platforms android --force
```

Cuidado: `--force` borra la carpeta de la plataforma y la vuelve a crear. Cualquier cambio hecho a mano dentro de `android/`, `ios/` o `web/` se pierde. Esa es justamente la razón de que no se versionen: los cambios deben vivir en `tool/bootstrap.py`, no en el resultado.

Atajos equivalentes por sistema:

```
./tool/bootstrap.sh --platforms android,web
```

```
.\tool\bootstrap.ps1 -platforms android,web
```

Variables de entorno

No identificadas. El código de la aplicación no lee ninguna variable de entorno: no hay `String.fromEnvironment`, `--dart-define` ni archivo `.env` en el repositorio. Toda la configuración de la aplicación vive en las preferencias del dispositivo y en la política de análisis inyectada por código.

Las únicas variables presentes son de la CI, declaradas en los propios workflows:

Variable	Dónde	Valor
FLUTTER_VERSION	los tres workflows	3.44.7
DEMO_BASE_HREF	deploy-landing.yml	/rootcause-qr-inspector/app/
GH_TOKEN	android-release.yml	El token efímero del propio workflow

Configuración inicial

No hay ninguna. La aplicación arranca con valores por defecto conservadores y crea su base de datos en el primer inicio. Detalle de cada preferencia y su valor por defecto en [10-configuration.md](#).

Creación o restauración de la base de datos

No hay ningún paso manual. La base se crea sola:

- `AppDatabase.open()` abre el archivo `rootcause_qr_inspector_v2.db` en el directorio de soporte de la aplicación (o el almacén IndexedDB del mismo nombre en web);
- `SchemaMigrator.migrate()` aplica los pasos de esquema pendientes dentro de una transacción, antes de que ningún repositorio la use.

No existe script SQL, ni servidor, ni credenciales de base de datos. Detalle en [07-database.md](#).

Para **restaurar** datos de un respaldo se usa la propia aplicación: Historial → menú → *Importar respaldo JSON*, que muestra una vista previa antes de tocar nada.

Ejecución en desarrollo

```
flutter run
```

Con un dispositivo o emulador conectado. Para elegir destino:

```
flutter devices  
flutter run -d <id-del-dispositivo>
```

La demo web:

```
flutter run -d chrome
```

En web no están disponibles la lectura de PDF ni el almacén seguro del sistema; la aplicación lo declara en la interfaz.

Ejecución en producción

No hay «servidor de producción». Producción es un artefacto instalable.

```
flutter build apk --release
```

El APK queda en `build/app/outputs/flutter-apk/app-release.apk`.

```
flutter build web --release
```

```
flutter build ios --release
```

El build de iOS requiere macOS y una identidad de firma. La CI solo compila el simulador (`flutter build ios --debug --simulator`), que no necesita firma.

Importante sobre la firma: el APK que publica la CI está firmado con la clave de depuración que genera Flutter, no con una clave comercial. El workflow lo verifica con `apksigner` y publica su SHA-256, pero eso **no** equivale a una firma de tienda. Ver [13-deployment-and-operations.md](#).

Ejecución de las pruebas

```
flutter test
```

Ejecuta los 87 casos de `test/`. Para cobertura:

```
flutter test --coverage
```

Un archivo concreto:

```
flutter test test/core/qr_investigation_engine_test.dart
```

La prueba de integración necesita un dispositivo:

```
flutter test integration_test/app_launch_test.dart
```

Verificaciones offline

Estas dos herramientas no necesitan Flutter y son la forma más rápida de saber si un cambio rompió un contrato del repositorio:

```
python tool/validate_structure.py --require-lock
```

Comprueba YAML y JSON válidos, imports locales existentes, enlaces Markdown locales que resuelven, acciones de GitHub fijadas a SHA, coherencia entre la versión de `pubspec.yaml` y la que muestra la interfaz, ausencia de rutas absolutas del entorno, ausencia de registro de contenido sensible, recursos de la landing, iconos y presencia del lockfile.

```
python tool/verify_rootcause_contract.py
```

Comprueba que las 26 reglas del motor existen también en el JSON Schema, en los textos de la interfaz y en la documentación de heurísticas; que los fixtures solo usan dominios reservados; que la política de ejemplo es sintética; y que el contrato de redacción de la evidencia sigue intacto.

En la CI, la primera se ejecuta con `uv` para no depender de un `pyyaml` instalado:

```
uv run --with pyyaml python tool/validate_structure.py --require-lock
```

Gate completo, igual que la CI

```
flutter pub get
python tool/bootstrap.py --platforms android,web
python tool/validate_structure.py --require-lock
python tool/verify_rootcause_contract.py
flutter analyze --fatal-infos
flutter test
flutter build web --release
flutter build apk --release
```

Hay dos scripts que encadenan variantes de esto:

Script	Para qué
<code>tool/run_quality_gate.sh</code>	Ciclo de trabajo diario: estructura, análisis, pruebas con cobertura, SBOM y licencias
<code>tool/finalize_stable.sh</code>	Antes de etiquetar una versión: exige Flutter en PATH, compila, regenera SBOM, licencias y <code>SOURCE_MANIFEST.json</code>

Errores frecuentes durante la instalación

Síntoma	Causa	Solución
No such file or directory: android/	No se ejecutó el bootstrap	<code>python tool/bootstrap.py --platforms android</code>
Flutter no está instalado o no está disponible en PATH	El bootstrap necesita Flutter para <code>flutter create</code>	Instalar Flutter y reabrir la terminal
cannot find symbol: class FilePickerPlugin	La plantilla generó AGP 9 y no se aplicaron los parches de la cadena Gradle	Volver a ejecutar el bootstrap, que fija AGP 8.11.1, KGP 2.2.20 y Gradle 8.14.3

Falta <code>pubspec.lock</code>	Se ejecutó <code>validate_structure.py --require-lock</code> sin resolver	<code>flutter pub get</code> y confirmar el lockfile
<code>ModuleNotFoundError: yaml</code>	<code>validate_structure.py</code> y <code>generate_sbom.py</code> necesitan <code>pyyaml</code>	<code>pip install pyyaml</code> o usar <code>uv run --with pyyaml</code>
Conflicto al resolver <code>pdfx</code> o <code>excel</code>	Ambas están ancladas por un conflicto real de <code>archive</code>	No subirlas sin leer ../quality/LOCKFILE.md
La versión mostrada en la interfaz no coincide	Se cambió <code>pubspec.yaml</code> sin cambiar <code>lib/core/app_info.dart</code>	Igualar ambos; es una comprobación deliberada
La demo web no lee PDF	El renderizador PDF solo existe en plataformas con <code>dart:io</code>	Exportar la página como imagen, o usar una plataforma nativa

Más síntomas, ya en ejecución, en [14-troubleshooting.md](#).

Herramientas auxiliares

Comando	Qué hace	Requiere
<code>python tool/generate_sbom.py</code>	SBOM CycloneDX en <code>build/reports/sbom.cdx.json</code>	<code>pyyaml</code> ; con Flutter incluye el árbol resuelto
<code>python tool/generate_license_inventory.py</code>	Inventario de licencias de las dependencias	<code>flutter pub get</code> previo
<code>python tool/generate_checksums.py <archivo...></code>	SHA-256 de cada archivo	—
<code>python tool/generate_source_manifest.py</code>	Regenera <code>SOURCE_MANIFEST.json</code> con el hash de cada archivo fuente	—
<code>python tool/generate_launcher_icons.py</code>	Regenera los iconos versionados	Pillow
<code>python tool/generate_scan_beep.py</code>	Regenera el tono de confirmación	—
<code>python tool/build_system_documentation.pdf.py</code>	Genera los PDF de esta documentación	<code>markdown</code> , <code>xhtml2pdf</code> ; <code>mmdc</code> para los diagramas